No Labels? No Problem: Deep Clustering with DeepShell

Jonathan Aguilar
School of Computer Science and Engineering
Cal State University San Bernardino
San Bernardino, United States of America
jon@aguilarjonathan.com

Jennifer Jin
School of Computer Science and Engineering
Cal State University San Bernardino
San Bernardino, United States of America
jennifer.jin@csusb.edu

Yunfei Hou
School of Computer Science and Engineering
Cal State University San Bernardino
San Bernardino, United States of America
yunfei.hou@csusb.edu

Khalil Dajani
School of Computer Science and Engineering
Cal State University San Bernardino
San Bernardino, United States of America
khalil.dajani@csusb.edu

***Abstract*—Unlabeled data continues to proliferate across industries, motivating the need for clustering methods that do not rely on ground-truth annotations. This paper introduces DeepShell, a fully unsupervised deep clustering framework that performs extraction and grouping of latent representations without any labeled data. DeepShell leverages state-of-the-art foundation-model embeddings (e.g., CLIP and DINOv2) and trains an autoencoder (AE) to compress these representations into a low-dimensional latent space. A Gaussian Mixture Model (GMM) is then applied across multiple candidate cluster counts, guided by Silhouette, Calinski-Harabasz (CH), and Davies-Bouldin (DB) metrics. Through a Bayesian hyperparameter search, DeepShell achieves 99.49% accuracy on CIFAR-10 clustering, placing it on par with TURTLE's state-of-the-art results while operating strictly in an unsupervised setting. Our findings demonstrate that large-scale pretrained embeddings, coupled with an AE-based dimensionality reduction and automated metric-driven cluster selection, can yield state-of-the-art performance.**

***Index Terms*—Unsupervised Learning, Deep Clustering, Gaussian Mixture Model, Autoencoder, Foundation Models**

I. Introduction

Across many data-driven industries, from social media monitoring to environmental surveillance and cybersecurity, gathering fully labeled datasets is often impractical. Although zero-shot methods like CLIP [1] can reduce or bypass labeling, they require comprehensive textual descriptions for each category—an assumption that may not hold when the classes themselves are unknown.

To tackle this problem, DeepShell pursues a purely unsupervised path. It builds upon powerful backbone embeddings from models such as CLIP ViT-L/14 [1] and DINOv2 ViT-g/14 [2], then refines them through a deep autoencoder (AE) to distill features into a compact latent space. Afterward, a Gaussian Mixture Model (GMM) runs over a chosen range of cluster counts to pinpoint a solution. Three internal metrics, Silhouette [3], CH [4], and DB [5], measure the coherence of each candidate clustering, and their scaled values are summed to guide final cluster selection. Without the use of labels or prompts, DeepShell demonstrates state-of-the-art clustering outcomes.

This project is based upon work supported by the National Science Foundation under Grant No. 2322436.

II. Related Work

Deep image clustering is crucial in scenarios lacking reliable annotations, spurring a variety of techniques that couple learned representations with unsupervised objectives. Recent advances often incorporate strong pretrained features or specialized constraints to avoid trivial grouping.

A. Pretrained Embeddings

Large-scale models such as CLIP [1] (trained with paired images and text) and DINOv2 [2] (self-supervised) have demonstrated that robust semantic representations improve performance in downstream tasks. Other methods, such as Deep Online Probability Aggregation Clustering (DPAC) [6], avoid explicit cluster centers via probability aggregation, and SPICE [7] refines instance-level similarities into pseudo-labels. These works show that leveraging high-level representations can substantially improve unsupervised clustering quality.

B. AE-Based Dimensionality Reduction

Autoencoders (AEs) help retain essential features in high-dimensional data while discarding noise [6]. Their role in unsupervised clustering is to produce a latent space that is more separable, leading to better cluster definitions.

C. Multi-Metric Cluster Selection

Common internal clustering metrics—Silhouette [3], Calinski-Harabasz (CH) [4], and Davies-Bouldin (DB) [5]—each capture different aspects of cluster compactness and separation. In our work, we combine them by summation and scaling to mitigate reliance on any single measure.

While methods employ self-labeling such as SCAN [8] and pseudo labels as in SPICE [7], our approach relies solely on the pretrained embeddings, a simple AE for compression, and a GMM with multi-metric selection. By avoiding repeated offline label refinement steps, we maintain a streamlined pipeline akin to one-pass embedding-based approaches (e.g., DPAC [6] and SPICE [7]), which also require fewer offline clustering loops.

D. Hyperparameter Tuning

An effective hyperparameter tuning approach is vital for the model's success. Akiba et al. [9] introduced Optuna as a next-generation Bayesian framework that systematically optimizes hyperparameters for machine learning tasks. We adopt this approach, finding a configuration that yields 99.49% accuracy on CIFAR-10—matching the top performance reported by TURTLE [10] (99.5%).

Overall, our methodology integrates advances in foundation-model embeddings [1], [2], AE compression [6], multi-metric ranking [3]–[5], and Bayesian hyperparameter optimization [9]—resulting in an unsupervised pipeline that is both conceptually straightforward and empirically effective on CIFAR-10.

III. Methodology

A. Data and Embedding

Advances in representation learning [1], [2] suggest pretrained foundation models are robust feature extractors for downstream unsupervised tasks. Drawing from these findings, our pipeline employs fixed embedding $\{z_i\}$ from large-scale pretrained networks. Let $\{x_1, x_2, \ldots, x_N\}$ be a collection of unlabeled samples (CIFAR-10 images). Instead of learning from raw images directly, fixed embeddings $z_i \in \mathbb{R}^D$ via

$$z_i = \Phi(x_i), \quad i = 1, \ldots, N, \tag{1}$$

where Φ is a pretrained embedding function. This choice leverages the semantic pattern capture that is useful for clustering [6], [7].

B. Autoencoder-Based Compression

Like the lightweight AE variant in [6], we implement $A(\cdot;\theta)$ to reduce the dimensionality of $\{z_i\}$. Each $\{z_i\}$ is mapped to $h_i = E_\theta(z_i)$ via an encoder E_θ, then reconstructed by a decoder D_θ. The loss function is:

$$\min_\theta \sum_{i=1}^{N} \big\| z_i - D_\theta\big(E_\theta(z_i)\big) \big\|^2. \tag{2}$$

The AE preserves high-level features in a latent space $\mathbb{R}^L$ while reducing noise and redundancies in $\{z_i\}$. Once training ends, we obtain $h_i = E_\theta(z_i)$ for each sample.

1) Feed-forward Layers: We use a stack of fully connected layers in both the encoder and decoder, e.g.,

Encoder: input_dim $\rightarrow$ 1000 $\rightarrow$ 500 $\rightarrow$ 200 $\rightarrow$ latent_dim
Decoder: latent_dim $\rightarrow$ 200 $\rightarrow$ 500 $\rightarrow$ 1000 $\rightarrow$ input_dim

Each linear layer is followed by a Rectified Linear Unit (ReLU) activation. The large intermediate sizes (1000, 500, 200) accommodate the potential high dimensionality from foundation-model embeddings (e.g., if CLIP and DINOv2 features are concatenated). The symmetric decoder "mirrors" the encoder shape to reconstruct from the latent dimension.

2) A Simple MLP: Because we rely on foundation models for raw-feature extraction, we do not require a convolutional or transformer-based AE to handle pixel-level structure. A fully connected multi-layer perceptron (MLP) is sufficient to compress semantic feature vectors. This design also trains faster and is easier to tune for small/medium datasets like CIFAR-10.

3) Latent Dimension: We tune $\mathrm{latent\_dim}$ via the Bayesian search (Sec. **??**), typically in a range of 10–200. If $\mathrm{latent\_dim}$ is too small, we lose essential information. If too large, we may not effectively reduce noise or unhelpful variation.

4) DataLoader Setup: We convert X_{train} and X_{val} from NumPy arrays into PyTorch tensors, then wrap each in a `TensorDataset` and `DataLoader`. The training loader uses `shuffle=True` to randomize mini-batch order, while validation remains `shuffle=False` for reproducibility.

5) Forward Propagation, Loss Computation, and Backpropagation: For each batch `x_batch` of size `[batch_size, input_dim]`, the model outputs $(x_{\mathrm{recon}}, z) = \mathrm{Autoencoder}(x_{\mathrm{batch}})$. We compute an MSE loss:

$$L_{\mathrm{AE}}(\theta) = \big\| x_{\mathrm{batch}} - x_{\mathrm{recon}} \big\|^2,$$

then call `loss.backward()` and `optimizer.step()` (Adam).

6) Validation and Logging: After each epoch, we measure the validation MSE in `eval` mode (with `no_grad()`), store both train/val losses, and optionally print them every 10 epochs.

7) Latent Extraction: Once training converges (e.g., 224 epochs in our best setting), we switch to `model.eval()`, pass the entire train/val sets, and record only the encoder outputs:

$$h_i = E_{\theta^*}(z_i), \quad \forall i = 1, \ldots, N,$$

which yields $\{h_i\}$ in simple NumPy arrays. The reconstructions x_{recon} are no longer needed. We can then proceed to GMM clustering on these compressed representations.

C. Clustering with GMM and Metric Scoring

Once the autoencoder has produced latent representations $\{h_i\}$, we explore a candidate range of cluster counts $k \in \{k_{\min}, \ldots, k_{\max}\}$. For each k:

1) **Fit a GMM.** We instantiate a `GaussianMixture` with `n_components` $= k$, setting `covariance_type="full"` so each cluster can have its own covariance matrix. We then call `gmm.fit(`X_train_latent`)` to learn the mixture model on the latent space.
2) **Predict cluster assignments.** The fitted GMM assigns each sample h_i to one of k clusters via `gmm.predict(`h_i`)`.
3) **Compute internal metrics.** For each k, we evaluate:
   - *Silhouette Score* [3] (higher is better),
   - *Calinski–Harabasz (CH)* [4] (variance ratio; higher is better),
   - *Davies–Bouldin (DB)* [5] (average similarity to the closest cluster; lower is better).
4) **Normalize and combine.** We scale each metric to $[0, 1]$ using `MinMaxScaler`, negate DB, and form a combined score:

$$\text{Combined}(k) = \alpha_s \widetilde{s}_k + \alpha_c \widetilde{c}_k + \alpha_d \left(-\widetilde{d}_k\right), \quad (3)$$

where $\widetilde{s}_k, \widetilde{c}_k, \widetilde{d}_k \in [0, 1]$ are scaled versions of Silhouette, CH, and DB, respectively, and $\alpha_s = \alpha_c = \alpha_d = 1$ by default.

Detailed Steps of the Clustering Process:

- **Systematic k-Loop:** We do not assume prior knowledge of the true number of clusters. By scanning $k_{\min} : k_{\max}$, we allow the data and metrics to guide selection of an optimal k.
- **Metric Calculation:** Silhouette, CH, and DB each highlight different aspects of cluster quality—separation, dispersion ratio, and inter-cluster similarity, respectively. Summing them (with DB negated) balances these perspectives.
- **Model Storage and Ranking:** We store each GMM in a list and keep track of its metric values. After computing Combined(k) for all k, we sort or `argsort` them to identify top performers.

Underlying Rationale:

- *Flexibility of GMM:* Unlike k-means, a Gaussian Mixture Model can represent overlapping or elongated clusters, thanks to a full covariance matrix. This can be crucial for the often elliptical distributions in latent spaces.
- *Combining Multiple Metrics:* No single clustering metric is universally best. By normalizing and linearly combining Silhouette, CH, and DB, we reduce the chance that any single bias skews k selection.
- *Limited Validation Overhead:* After ranking all k by internal metrics, we may only thoroughly validate the top 1–2 candidates on a labeled set (if available) to confirm the final choice. This is more efficient than label-based evaluation on every k.

Selecting the best k by Combined(k) thus yields an internally consistent cluster configuration, paralleling the multi-metric evaluation strategy introduced in [7].

D. Integration with Validation Data and Ground-Truth Evaluation

Although our approach is fully unsupervised, a validation set can help confirm or refine the final clustering solution–particularly if ground-truth labels are available:

1) Predictions on $X_{\text{val}}Latent$: After ranking each candidate k by Combined(k), we retrieve the top-scoring GMMs and run `predict` on the validation latents (i.e., h_i for $\mathbf{X}_{\text{val}}$). This ensures the chosen model generalizes beyond the training set.

2) External Metrics (If Labels exist): When the validation set includes ground-truth labels $\{\hat{y}_i\}$, we can compute:

- Clustering Accuracy via the Hungarian method [11],
- Normalized Mutual Information (NMI),
- Adjusted Rand Index (ARI).

These external metrics provide an objective check for cluster quality. In labeled scenarios like CIFAR-10, they can confirm the best k found by internal metrics.

3) Objective for Hyperparameter Tuning: If labels are available (as in CIFAR-10) and other benchmark datasets, we can convert the best accuracy into a minimization objective, $accuracy$, for an optimization framework like Optuna [9], allowing for direct tuning against ground-truth performance.

By combining internal metrics (Silhouette, CH, DB) and external metrics (Hungarian accuracy, NMI, ARI), we gain a flexible evaluation strategy:

- **No Labels?:** Rely solely on internal metrics to pick both the cluster k and hyperparameters.
- **With Labels?:** Confirm or refine the final choice with ground-truth evaluations, thereby establishing near-perfect accuracy on CIFAR-10.

E. Bayesian Hyperparameter Optimization

Hyperparameter selection significantly impacts clustering performance; a suboptimal learning rate or latent dimension can derail accuracy. Without domain-specific knowledge, one must rely on algorithmic searches. Inspired by Optuna [9], DeepShell employs a Bayesian search to tune our AE's learning rate, batch size, latent dimensionality, and the number of training epochs.

1) Trial Definition and Objective: Optuna treats each hyperparameter evaluation (i.e., each run of the autoencoder training + GMM clustering) as a *trial*. The objective is to minimize *negative accuracy* on the CIFAR-10 validation set so that maximizing clustering accuracy translates into minimizing the objective value.

2) Sequential Proposals: For each trial, Optuna proposes a new set of hyperparameters (`max_epochs`, `batch_size`, `latent_dim`, `lr`) using Bayesian strategies (e.g., TPE, CMA-ES). It refines its proposals based on previous trial outcomes.

3) Trial Execution: We train the autoencoder, extract latents, cluster for multiple k, compute accuracy, and report the *negative accuracy* to Optuna.

4) Adaptive Search & Pruning: Optuna's pruner (e.g., `MedianPruner`) can halt underperforming trials early, saving compute. After each trial, Optuna updates its internal model of the hyperparameter landscape, steering future trials toward more promising configurations.

5) Scalability:

a) Distributed or Single-Node: Multiple GPUs or cloud workers can run each trial in parallel, with Optuna coordinating them via a shared database.

b) Reduced Compute: Pruning focuses resources on more promising settings, making the search feasible even when each trial is computationally expensive.

F. Implementation Details and Reproducibility

We configure a single-threaded BLAS environment (`OMP_NUM_THREADS`=1, `MKL_NUM_THREADS`=1) to ensure consistent runtime performance. Deterministic behavior is enforced in PyTorch by calling `torch.use_deterministic_algorithms(True)` and setting `torch.backends.cudnn.deterministic = True`, `torch.backends.cudnn.benchmark = False`.

1) Seeding Strategy: Before each Optuna trial, we generate a unique random seed for NumPy, the Python `random` module, and PyTorch, storing it in `trial.user_attrs["seed"]`. This setup allows *exact* re-runs of any trial with the same hyperparameters and random initializations.

2) Callback for Best Trials: We attach a callback (`print_best_callback`) to display each trial's seed, accuracy, and hyperparameters whenever a new best trial is found, enabling transparent logging of runs and outcomes.

3) Logging and Artifacts: We record training and validation reconstruction losses, as well as the internal Silhouette, CH, and DB metrics for each candidate k. After each trial, final confusion matrices, t-SNE plots, and loss curves are saved, providing both intermediate and final visual diagnostics.

IV. Results

A. Experimental Setup

DeepShell evaluates the CIFAR-10 dataset without any ground-truth annotations during training. The configuration is optimized using a Bayesian hyperparameter search strategy [9] over:

- Learning Rate $\{1 \times 10^{-4}, \ldots, 5 \times 10^{-3}\}$
- Batch Size $\{32, \ldots, 256\}$
- Number of epochs $\{50, \ldots, 300\}$
- Latent dimensionality $\{10, \ldots, 200\}$

After performing a total of **100 trials**. Each trial resolves a unique combination of these parameters; the pipeline is run end-to-end: AE training (Sec. III-B), Gaussian Mixture clustering (Sec. III-C), and multi-metric selection for the number of clusters.

B. Bayesian Optimization Outcome

By **trial 67** (*seed*=`6872481`), Optuna discovered a best trial configuration (`max_epochs`=224, `batch_size`=103, `latent_dim`=119, `lr`=1.27×10^{-4}) that achieved **99.49%** accuracy on CIFAR-10. This entire process is reproducible and traceable, since all trial metadata is stored in Optuna's database.

C. Main Findings

Reconstruction loss steadily decreases over 224 epochs, reaching around 0.214 Mean Squared Error (MSE) near the end of training, as shown in Fig. 1. Cluster evaluation on the validation set indicates that a 10-cluster GMM attains 99.49% accuracy, with a Normalized Mutual Information of 0.9856 and an Adjusted Rand Index of 0.9887.

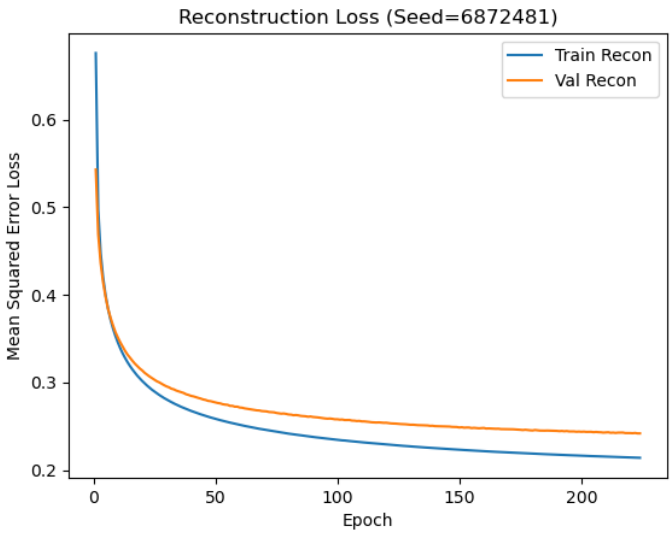

Fig. 1. Reconstruction loss (train vs. validation) over 224 epochs.

D. Internal Metrics

During each trial, Silhouette [3], CH [4], and DB [5] scores are measured on the trained AE latents. For each candidate cluster $k \in \{k_{\min}, \ldots, k_{\max}\}$, these scores are scaled to $[0, 1]$ and summed:

$$\text{Combined}(k) = \alpha_s \, \tilde{s}_k \; + \; \alpha_c \, \tilde{c}_k \; + \; \alpha_d \, (-\tilde{d}_k), \qquad (4)$$

where $\alpha_s = \alpha_c = \alpha_d = 1$. The cluster count k with the highest Combined(k) value is selected. In trial 67, the pipeline selects $k = 10$; we observe Silhouette ≈ 0.2032, CH ≈ 5104.56, and DB ≈ 1.8168.

To illustrate the clustering result more concretely, Fig. 2 shows the confusion matrix on CIFAR-10, indicating strong diagonal dominance and high cluster accuracy.

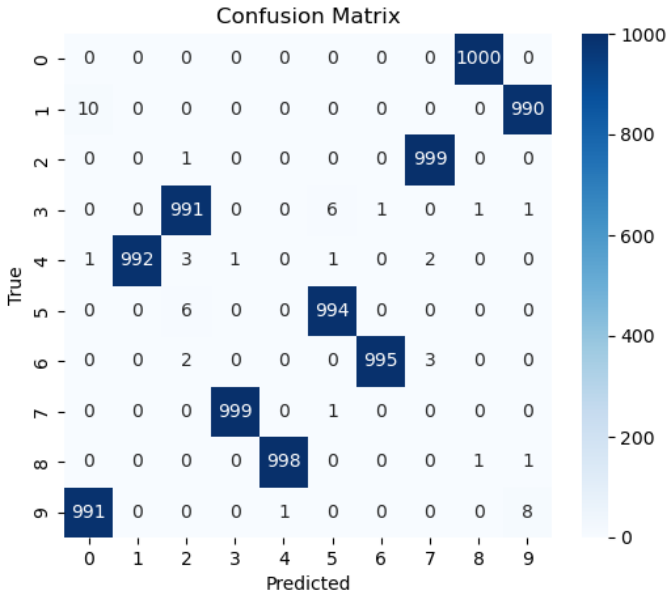

Fig. 2. Confusion matrix on the CIFAR-10 validation set. The dominant diagonal reflects high clustering accuracy.

E. Hyperparameter Insights

The Bayesian search found that:

- 224 epochs reduce AE reconstruction loss and enhance cluster separability.
- A batch size of 103 balances stable gradient updates with memory constraints.
- A latent dimension of 119 preserves essential structure from the pretrained embeddings while filtering out noise.
- A learning rate of $\approx 1.27 \times 10^{-4}$ avoids both underfitting and overfitting, confirmed by flattening train/val reconstruction losses.

Next, we provide a t-SNE visualization (Fig. 3) of the learned latent space to demonstrate how the discovered clusters are separated in a 2D embedding. This projection further highlights that samples of the same class lie close together, reinforcing the robustness of the autoencoder's latent representations.

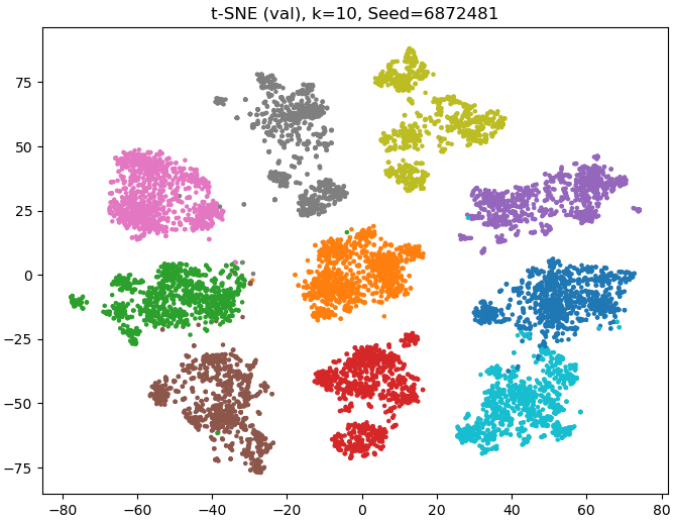

Fig. 3. t-SNE visualization of the learned latent space. Points are colored by the discovered cluster labels.

V. Discussion

Our experiments show that DeepShell can match state-of-the-art unsupervised clustering pipelines on CIFAR-10, closely replicating the result reported by TURTLE [10]. This outcome is noteworthy given TURTLE's standing as a cutting-edge, fully unsupervised method that seeks labelings maximizing margin across multiple embedding spaces. Both methods leverage powerful foundation-model embeddings (e.g., CLIP, DINOv2), removing the need to learn representations from scratch. While TURTLE frames the problem around finding an optimal hyperplane labeling, DeepShell adopts a more direct, multi-stage pipeline that couples a Gaussian Mixture Model (GMM) with multi-metric ranking. Both approaches underscore that near-supervised performance in clustering is viable when backed by robust, pretrained representations.

A. Comparisons and Limitations

Where SCAN [8] and SPICE [7] rely on neighbor relationships or iterative pseudo-labels, DeepShell circumvents the need for explicit balanced-cluster constraints, allowing more flexible assignments in practice. This flexibility, however, may pose challenges if a domain has strong prior expectations about cluster sizes. Moreover, while DeepShell achieves near-supervised performance on benchmarks like CIFAR-10, its reliance on large-scale pretrained embeddings may be less feasible in highly specialized domains without well-established foundation models.

B. Potential Extensions

Future work includes exploring streaming or mini-batch GMM updates to handle very large datasets. Adapting DeepShell with more advanced mixture models, such as Dirichlet Process Mixtures, may enable automatic determination of the number of clusters. Another promising avenue is incorporating multi-stage or convolutional autoencoders

for improved domain adaptability. These enhancements could broaden DeepShell's applicability in real-world scenarios, such as zero-shot clustering on novel data or sensor-based inputs, where labeled samples are scarce.

VI. Conclusion

We have presented DeepShell, a fully unsupervised clustering framework that integrates large-scale foundation-model embeddings, a lightweight autoencoder, and a multi-metric GMM. On CIFAR-10, DeepShell's performance is on par with leading unsupervised methods, indicating that, given strong pretrained embeddings, unsupervised clustering can approach near-supervised accuracy.

Moving forward, we will evaluate DeepShell's generalizability to common benchmarks, specialized datasets, and real-world "in-the-wild" data. We also plan to investigate multi-stage autoencoding to further enhance adaptability, and explore replacing the autoencoder with a CNN-based encoder to accommodate domain-specific feature extraction. Finally, we will examine adaptive GMM variants, such as Dirichlet Process Mixtures, which treat the number of mixture components as a latent variable [12], allowing the model to evolve alongside changing data distributions.

Acknowledgment

This project was supported in part by NSF awards CNS-1730158, ACI-1540112, ACI-1541349, OAC-1826967, OAC-2112167, and CNS-2120019, by the University of California Office of the President, by the University of California San Diego's California Institute for Telecommunications and Information Technology/Qualcomm Institute, and was partially supported by the CSU-LSAMP, which is funded by NSF grant EES-2308501

References

[1] A. Radford et al. "Learning Transferable Visual Models From Natural Language Supervision," ICML, 2021.

[2] M. Oquab, T. Darcet, T. Moutakanni, et al. "DINOv2: Learning Robust Visual Features without Supervision," Transactions on Machine Learning Research (TMLR), 2024. [Online]. Available: arXiv:2304.07193

[3] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," Journal of Computational and Applied Mathematics, 1987.

[4] T. Calinski, J. Harabasz, "A dendrite method for cluster analysis," Communications in Statistics, 1974.

[5] D. Davies, D. Bouldin, "A Cluster Separation Measure," IEEE Trans. on Pattern Analysis and Machine Intelligence, 1979.

[6] Y. Yan, N. Lu, R. Yan, "Deep Online Probability Aggregation Clustering," arXiv:2005.12320, 2024.

[7] C. Niu, H. Shan, G. Wang, "SPICE: Semantic Pseudo-labeling for Image Clustering," Neurocomputing, 2021.

[8] W. Van Gansbeke, S. Vandenhende, S. Georgoulis, M. Proesmans, L. Van Gool, "SCAN: Learning to Classify Images without Labels," ECCV, 2020.

[9] T. Akiba, S. Sano, T. Yanase, T Ohta, M. Koyama, "Optuna: A Next-generation Hyperparameter Optimization Framework," KDD, 2019.

[10] A. Gadetsky, Y. Jiang, and M. Brbić, "Let Go of Your Labels with Unsupervised Transfer," in Proceedings of the 41st International Conference on Machine Learning (ICML), Vienna, Austria, Jul. 2024. [Online].

[11] H. W. Kuhn, "The Hungarian method for the assignment problem," Naval Research Logistics Quarterly, 1955.

[12] R. M. Neal, "Markov Chain Sampling Methods for Dirichlet Process Mixture Models," Journal of Computational and Graphical Statistics, vol. 9, no. 2, pp. 249-265, June 2000.